보안 계층 분석

개요

이 문서는 Rust libp2p의 보안 계층 구현을 상세히 분석한 결과를 제공합니다. libp2p는 Noise 프로토콜과 TLS를 통해 피어 간 안전한 통신을 보장하며, 각 프로토콜은 고유한 특징과 장점을 가지고 있습니다.

Noise 프로토콜 분석

개요 및 특징

Noise 프로토콜은 libp2p의 주요 보안 프로토콜로, 다음과 같은 특징을 가집니다:

- **모듈식 설계**: 다양한 핸드셰이크 패턴 지원 (XX, IK, IX)
- **전방향 보안성**: 세션 키 노출 시에도 과거 통신 보호
- **상호 인증**: 양방향 피어 신원 확인
- **효율성**: 최소한의 라운드트립으로 보안 채널 설정

핵심 구성 요소

Config 구조체

```
#[derive(Clone)]
pub struct Config {
dh_keys: AuthenticKeypair, // DH 키 쌍
params: NoiseParams, // Noise 매개변수
```

```
webtransport_certhashes: Option<HashSet<Multihash<64>>>, // WebTransport 인증서 ㅎ
prologue: Vec<u8>, // 핸드셰이크 프롤로그
}
```

주요 필드 설명:

- `dh_keys` : X25519 Diffie-Hellman 키 쌍과 libp2p 신원 키의 조합
- `params` : XX 핸드셰이크 패턴 매개변수
- `webtransport_certhashes` : WebTransport 연결 시 인증서 검증용
- `prologue` : 핸드셰이크에 포함될 추가 데이터

AuthenticKeypair 구조체

```
pub struct AuthenticKeypair {
keypair: Keypair, // X25519 DH 키 쌍
identity: KeypairIdentity, // libp2p 신원과 서명
}

pub struct KeypairIdentity {
public: identity::PublicKey, // libp2p 공개 키
signature: Vec<u8>, // DH 공개 키에 대한 서명
}
```

인증 메커니즘:

1. X25519 DH 키 쌍 생성

2. libp2p 신원 키로 DH 공개 키에 서명

3. 핸드셰이크 중 서명 검증으로 신원 확인

XX 핸드셰이크 패턴

libp2p는 주로 XX 핸드셰이크 패턴을 사용하며, 이는 다음과 같은 특징을 가집니다:

```
Initiator Responder

--------- ---------

-> e

<- e, ee, s, es

-> s, se
```

메시지 흐름:

1. **Message 1**: Initiator가 임시 키(e) 전송

2. **Message 2**: Responder가 임시 키(e), 키 교환(ee), 정적 키(s), 키 교환(es) 전송

3. **Message 3**: Initiator가 정적 키(s), 키 교환(se) 전송

핸드셰이크 구현 상세

State 구조체

```rust
pub(crate) struct State<T> {

io: Framed<T, Codec<snow::HandshakeState>>, // 프레임된 I/O

identity: KeypairIdentity, // 로컬 신원

dh_remote_pubkey_sig: Option<Vec<u8>>, // 원격 DH 키 서명

id_remote_pubkey: Option<identity::PublicKey>, // 원격 공개 키
```

```
responder_webtransport_certhashes: Option<HashSet<Multihash<64>>>, // WebTranspo
remote_extensions: Option<Extensions>, // 원격 확장
}
```

핸드셰이크 메시지 처리

빈 메시지 전송/수신:

```
pub(crate) async  fn send_empty<T>(state: &mut State<T>) -> Result<(), Error>
where
T: AsyncWrite + Unpin,
{
state
.io
.send(&proto::NoiseHandshakePayload::default())
.await?;
Ok(())
}
```

```
pub(crate) async  fn recv_empty<T>(state: &mut State<T>) -> Result<(), Error>
where
T: AsyncRead + Unpin,
{
let payload = recv(state).await?;
```

```
if payload.get_size() != 0 {
return Err(io::Error::new(io::ErrorKind::InvalidData, "Expected empty payload.")
}
Ok(())
}
```

신원 정보 전송/수신:

```
pub(crate) async  fn send_identity<T>(state: &mut State<T>) -> Result<(), Error>
where
T: AsyncRead + AsyncWrite + Unpin,
{
let  mut pb = proto::NoiseHandshakePayload {
identity_key: state.identity.public.encode_protobuf(),
..Default::default()
};


pb.identity_sig.clone_from(&state.identity.signature);


// Responder인 경우 WebTransport 인증서 해시 전송
if state.io.codec().is_responder() {
if  let Some(ref certhashes) = state.responder_webtransport_certhashes {
let ext = pb.extensions.get_or_insert_with(proto::NoiseExtensions::default);
ext.webtransport_certhashes = certhashes.iter().map(|hash| hash.to_bytes()).coll
```

```
}
}

state.io.send(&pb).await?;
Ok(())
}

pub(crate) async  fn recv_identity<T>(state: &mut State<T>) -> Result<(), Error>
where
T: AsyncRead + Unpin,
{
let pb = recv(state).await?;
state.id_remote_pubkey = Some(identity::PublicKey::try_decode_protobuf(&pb.ident

if !pb.identity_sig.is_empty() {
state.dh_remote_pubkey_sig = Some(pb.identity_sig);
}

if  let Some(extensions) = pb.extensions {
state.remote_extensions = Some(extensions.into());
}

Ok(())
```

```
}
```

인바운드/아웃바운드 업그레이드

인바운드 업그레이드 (Responder)

```
impl<T> InboundConnectionUpgrade<T> for Config

where

T: AsyncRead + AsyncWrite + Unpin + Send + 'static,

{

type Output = (PeerId, Output<T>);

type Error = Error;

type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S


fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {

async  move {

let  mut state = self.into_responder(socket)?;


// XX 패턴 핸드셰이크 수행

handshake::recv_empty(&mut state).await?; // Message 1 수신

handshake::send_identity(&mut state).await?; // Message 2 전송

handshake::recv_identity(&mut state).await?; // Message 3 수신
```

```
let (pk, io) = state.finish()?;

Ok((pk.to_peer_id(), io))

}

.boxed()

}

}
```

아웃바운드 업그레이드 (Initiator)

```
impl<T> OutboundConnectionUpgrade<T> for Config

where

T: AsyncRead + AsyncWrite + Unpin + Send + 'static,

{

type Output = (PeerId, Output<T>);

type Error = Error;

type Future = Pin<Box<dyn Future<Output = Result<Self::Output, Self::Error>> + S


fn upgrade_outbound(self, socket: T, _: Self::Info) -> Self::Future {

async  move {

let  mut state = self.into_initiator(socket)?;


// XX 패턴 핸드셰이크 수행

handshake::send_empty(&mut state).await?; // Message 1 전송

handshake::recv_identity(&mut state).await?; // Message 2 수신
```

```
handshake::send_identity(&mut state).await?; // Message 3 전송


let (pk, io) = state.finish()?;

Ok((pk.to_peer_id(), io))

}

.boxed()

}

}
```

핸드셰이크 완료 및 검증

```
pub(crate) fn finish(self) -> Result<(identity::PublicKey, Output<T>), Error> {

let is_initiator = self.io.codec().is_initiator();

let (pubkey, framed) = map_into_transport(self.io)?;


let id_pk = self

.id_remote_pubkey

.ok_or_else(|| Error::AuthenticationFailed)?;


// DH 공개 키에 대한 서명 검증

let is_valid_signature = self.dh_remote_pubkey_sig.as_ref().is_some_and(|s| {

id_pk.verify(&[STATIC_KEY_DOMAIN.as_bytes(), pubkey.as_ref()].concat(), s)

});
```

```
if !is_valid_signature {
return Err(Error::BadSignature);
}

// WebTransport 인증서 해시 검증 (Initiator인 경우)
if is_initiator {
if  let Some(expected_certhashes) = self.responder_webtransport_certhashes {
let ext = self.remote_extensions.ok_or_else(|| {
Error::UnknownWebTransportCerthashes(expected_certhashes.to_owned(), HashSet::ne
})?;

let received_certhashes = ext.webtransport_certhashes;

// 예상 인증서 해시가 수신된 해시의 부분집합이어야 함
if !expected_certhashes.is_subset(&received_certhashes) {
return Err(Error::UnknownWebTransportCerthashes(
expected_certhashes,
received_certhashes,
));
}
}
}
```

```
Ok((id_pk, Output::new(framed)))
}
```

검증 과정:

1. **서명 검증**: 원격 피어의 libp2p 키로 DH 공개 키 서명 확인
2. **신원 확인**: 검증된 공개 키로 PeerId 생성
3. **WebTransport 검증**: 필요시 인증서 해시 일치 확인
4. **암호화 채널**: 검증 완료 후 암호화된 통신 채널 반환

TLS 구현 분석

개요 및 특징

libp2p-tls는 표준 TLS 1.3을 기반으로 하며 다음과 같은 특징을 가집니다:

- **표준 호환성**: 기존 TLS 인프라와 호환
- **성숙한 프로토콜**: 검증된 보안 프로토콜
- **인증서 기반**: X.509 인증서를 통한 신원 확인
- **libp2p 확장**: libp2p 특화 확장으로 PeerId 매핑

주요 의존성

```
[dependencies]
rustls = { version = "0.23.9", features = ["ring", "std"] }
futures-rustls = { workspace = true }
```

```
rcgen = { workspace = true } # 인증서 생성

ring = { workspace = true } # 암호화 라이브러리

webpki = { version = "0.103", package = "rustls-webpki" }

x509-parser = "0.17.0" # X.509 파싱

yasna = "0.5.2" # ASN.1 인코딩
```

인증서 생성 및 검증

자체 서명 인증서 생성:

```
// libp2p 신원 키를 기반으로 X.509 인증서 생성

fn generate_certificate(identity: &identity::Keypair) -> Result<Certificate, Err

let  mut params = CertificateParams::new(vec!["libp2p".to_string()]);

// libp2p 공개 키를 Subject Alternative Name에 포함

params.subject_alt_names = vec![

SanType::Extension(

// libp2p PeerId를 ASN.1 확장으로 인코딩

encode_peer_id_extension(identity.public().to_peer_id())?

)

];

// 키 쌍 설정

params.key_pair = Some(convert_keypair(identity)?);

Certificate::from_params(params)
```

```
}
```

인증서 검증:

```
fn verify_certificate(cert: &Certificate, expected_peer_id: Option<PeerId>) -> R

// 1. 기본 X.509 검증

verify_basic_constraints(cert)?;

// 2. libp2p 확장에서 PeerId 추출

let peer_id = extract_peer_id_from_extension(cert)?;

// 3. 예상 PeerId와 일치 확인

if  let Some(expected) = expected_peer_id {

if peer_id != expected {

return Err(Error::PeerIdMismatch);

}

}

// 4. 공개 키와 PeerId 일치 확인

let public_key = extract_public_key(cert)?;

if public_key.to_peer_id() != peer_id {

return Err(Error::InvalidCertificate);

}

Ok(peer_id)

}
```

보안 프로토콜 비교

Noise vs TLS

| 특징 | Noise | TLS |

|------|-------|-----|

| **설계 철학** | 최소주의, 모듈식 | 포괄적, 표준화 |

| **핸드셰이크** | 3 메시지 (XX 패턴) | 1-RTT (TLS 1.3) |

| **인증서** | 불필요 | X.509 인증서 |

| **확장성** | 높음 (패턴 선택) | 중간 (확장 메커니즘) |

| **구현 복잡도** | 낮음 | 높음 |

| **상호 운용성** | libp2p 특화 | 범용 |

| **성능** | 높음 | 중간 |

| **전방향 보안성** | 기본 제공 | 기본 제공 |

사용 시나리오

Noise 프로토콜 선택 시:

- P2P 네트워크 내부 통신
- 최소한의 오버헤드 필요
- 빠른 핸드셰이크 중요
- libp2p 생태계 내 사용

TLS 선택 시:

- 기존 인프라와 호환성 필요
- 표준 준수 요구사항

- 웹 브라우저와의 상호 운용성
- 엔터프라이즈 환경

암호화 및 키 관리

키 유형 및 용도

```
// libp2p 신원 키 (장기 사용)
pub enum Keypair {
Ed25519(ed25519::Keypair), // 기본 권장
Rsa(rsa::Keypair), // 레거시 지원
Secp256k1(secp256k1::Keypair), // 블록체인 호환
Ecdsa(ecdsa::Keypair), // NIST 곡선
}

// Noise DH 키 (세션별 생성)
pub struct Keypair {
secret: [u8; 32], // X25519 비밀 키
public: [u8; 32], // X25519 공개 키
}
```

키 교환 과정

Noise 프로토콜:

1. **정적 키**: libp2p 신원 키 (장기 사용)

2. **임시 키**: 세션별 X25519 키 쌍

3. **키 유도**: HKDF를 통한 세션 키 생성

4. **전방향 보안성**: 임시 키 삭제로 보장

TLS 프로토콜:

1. **인증서 키**: X.509 인증서의 공개 키

2. **ECDHE**: 임시 키 교환

3. **키 유도**: TLS 1.3 키 스케줄

4. **전방향 보안성**: 임시 키 삭제로 보장

보안 고려사항

위협 모델

보호 대상:

- 통신 내용의 기밀성
- 메시지 무결성
- 피어 신원 인증
- 재생 공격 방지

가정:

- 네트워크는 신뢰할 수 없음
- 중간자 공격 가능
- 트래픽 분석 가능
- 일부 키 노출 가능

보안 속성

기밀성 (Confidentiality):

- ChaCha20-Poly1305 (Noise) 또는 AES-GCM (TLS) 암호화
- 세션별 고유 키 사용
- 키 로테이션 지원

무결성 (Integrity):

- 인증된 암호화 (AEAD) 사용
- 메시지 순서 보장
- 중복 방지

인증 (Authentication):

- 디지털 서명을 통한 신원 확인
- 공개 키 기반 인증
- PeerId와 키 바인딩

전방향 보안성 (Forward Secrecy):

- 임시 키 교환 사용
- 세션 종료 시 키 삭제
- 과거 통신 보호

성능 최적화

Noise 프로토콜 최적화

```
// 키 재사용을 통한 성능 향상
pub struct Config {
dh_keys: AuthenticKeypair, // 재사용 가능한 DH 키
```

```
// ...
}

// 비동기 처리로 블로킹 방지
async fn upgrade_outbound(self, socket: T, _: Self::Info) -> Self::Future {
// 논블로킹 핸드셰이크
}
```

메모리 효율성

- **Zero-copy**: 가능한 경우 데이터 복사 최소화
- **스트림 암호화**: 청크 단위 처리로 메모리 절약
- **키 관리**: 안전한 키 삭제 (zeroize 크레이트 사용)

결론

Rust libp2p의 보안 계층은 다음과 같은 특징을 가집니다:

1. **이중 보안 옵션**: Noise와 TLS 모두 지원으로 다양한 요구사항 충족
2. **강력한 암호화**: 현대적 암호화 알고리즘과 프로토콜 사용
3. **전방향 보안성**: 임시 키 교환으로 장기 보안 보장
4. **효율적 구현**: 비동기 처리와 최적화된 암호화 라이브러리
5. **확장 가능성**: WebTransport 등 새로운 프로토콜 지원

이러한 설계는 P2P 네트워크에서 요구되는 높은 보안 수준을 제공하면서도 성능과 사용성을 균형있게 유지합니다.